

BAŞLIK: REST'İN ÖTESİNDE: GRAPHQL, GRPC VE WEBHOOK MİMARİLERİ

TÜR: Akademik/Teknik Detaylı Anlatım

BÖLÜM 1: REST'İN SINIRLARI — NEDEN ALTERNATİFLERE İHTİYAÇ DUYULDU?

REST (bkz. 22.3), basitliği ve HTTP ile doğal uyumu sayesinde en yaygın API mimarisi olmayı sürdürüyor. Ama belirli senaryolarda REST'in doğasından kaynaklanan iki sorun ortaya çıkar:

- Aşırı Veri Çekme (Over-fetching):** Bir mobil uygulama sadece bir kullanıcının adını ve profil fotoğrafını göstermek istese bile, GET /kullanicilar/42 isteği o kullanıcının TÜM alanlarını (adres, telefon, kayıt tarihi, tercihleri...) döner — kullanılmayan veri, özellikle mobil ağlarda gereksiz bant genişliği tüketir.
- Yetersiz Veri Çekme (Under-fetching):** Bir ekranda hem kullanıcı bilgisi hem son 5 siparişi hem de her siparişin ürünleri gösterilecekse, REST ile bu genellikle 3 AYRI istek gerektirir (/kullanicilar/42, sonra /kullanicilar/42/siparisler, sonra her sipariş için ayrı ürün istekleri) — bu da gecikmeyi (latency) katlar.

GraphQL ve gRPC, bu sorunlara farklı açılardan çözüm üretmek için doğmuş mimarilerdir; her ikisi de REST'in "yerine geçmek" için değil, FARKLI kullanım senaryolarında daha uygun olmak için tasarlanmıştır.

BÖLÜM 2: GRAPHQL

GraphQL, Facebook tarafından 2012'de geliştirilen (2015'te açık kaynak yapılan), bir SORGU DİLİDİR. REST'teki "her kaynak için ayrı URL" yaklaşımının aksine, GraphQL API'leri genellikle TEK bir endpoint sunar (POST /graphql) ve istemci, TAM OLARAK hangi alanları istediğini bir SORGU olarak gövdede belirtir:

```
sorgu { kullanıcı(id: 42) { ad, profilFoto, siparisler { urunler { isim, fiyat } } } }
```

Sunucu, TAM OLARAK istenen alanları, TEK bir istekte, iç içe (nested) bir JSON olarak döner — bu hem over-fetching hem under-fetching sorununu aynı anda çözer. Bedeli: (1) sunucu tarafında daha karmaşık bir çözümleme (resolver) katmanı gerekir, (2) HTTP seviyesinde önbellekleme (bkz. 22.2, Bölüm 5, Cache-Control) doğrudan çalışmaz çünkü her sorgu farklı bir POST gövdesidir — GraphQL istemcileri (örn. Apollo Client) kendi önbellekleme katmanlarını uygular.

GraphQL, özellikle karmaşık, iç içe geçmiş veri ilişkileri olan ve çok sayıda farklı istemci türünün (web, mobil, akıllı TV) FARKLI veri ihtiyaçları olduğu ürünlerde (örn. GitHub, Shopify, Facebook) tercih edilir.

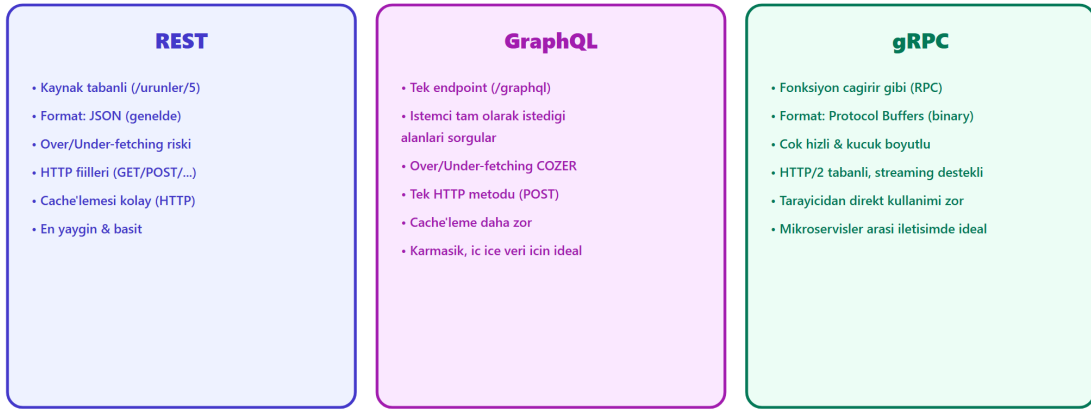
BÖLÜM 3: GRPC

gRPC, Google tarafından geliştirilen, RPC (Remote Procedure Call - Uzak Prosedür Çağrısı) prensibine dayanan bir çerçevedir: istemci, sanki sunucudaki bir fonksiyonu YEREL olarak çağırıyormuş gibi kod yazar (stub.KitapGetir(id=42)), ama arka planda bu bir ağ isteğine dönüşür. gRPC'nin temel teknik farkları:

1. Protocol Buffers (Protobuf): Veri, JSON gibi okunabilir metin yerine, Google'ın geliştirdiği İKİLİ (binary) bir formatla serileştirilir — hem çok daha küçük boyutludur hem de serileştirme/ayırıştırma çok daha hızlıdır.
2. HTTP/2 Tabanlı: Tek bir bağlantı üzerinden çoklu, eşzamanlı istek/yanıt akışını (multiplexing) ve gerçek zamanlı, çift yönlü akış (streaming) desteğini doğal olarak sağlar.
3. Sıkı Şema (Strict Schema): İstek/yanıt yapıları .proto dosyalarında tanımlanır; bu dosyalardan onlarca dilde (Python, Go, Java, C++...) istemci/sunucu kodu OTOMATİK üretilebilir — büyük, çok-dilli mikroservis filolarında (bkz. 22.9) inanılmaz değerlidir.

Dezavantajı: İkili format ve özel protokol nedeniyle bir web tarayıcısından DOĞRUDAN kullanılamaz (gRPC-Web adlı bir köprü katmanı gerekir) — bu yüzden gRPC, kullanıcıya dönük (public-facing) API'lerde değil, ŞİRKET İÇİ mikroservisler arası iletişimde (servis-servis haberleşmesi) yaygın olarak tercih edilir.

REST vs GraphQL vs gRPC

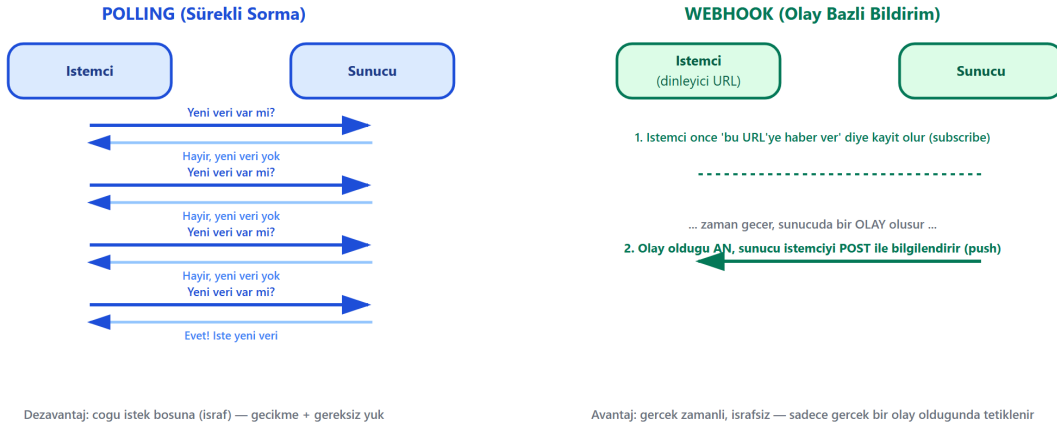


Şekil 1 — REST, GraphQL ve gRPC'nin karşılaştırması: her biri farklı bir kullanım senaryosunda güçlüdür, birbirinin "yerine" değil "tamamlayıcısı"dır.

BÖLÜM 4: WEBHOOK'LAR — API'NİN TERSİ YÖNDE ÇALIŞMASI

Şimdiye kadar incelediğimiz her şey "istemci sorar, sunucu cevaplar" modelindeydi (pull model). Ama bazı senaryolarda, bir OLAYIN NE ZAMAN olacağını istemci bilemez — "ödeme ne zaman tamamlanacak?", "kargo ne zaman teslim edilecek?". Bu durumda istemcinin sürekli sorup durması (polling) israftır. Webhook, bu ilişkiyi TERSİNE ÇEVİRİR: istemci önce "bir olay olduğunda beni bu URL'den haberdar et" diye KAYIT OLUR (subscribe); olay gerçekleştiğinde SUNUCU, istemcinin verdiği URL'ye bir HTTP POST isteği GÖNDERİR (push model).

Polling vs Webhook: Güncel Veriyi Almanın İki Yolu



Şekil 2 — Polling'de istemci sürekli sorar (çoğu cevap israf); webhook'ta sunucu sadece gerçek bir olay olduğunda haber verir.

4.1. Webhook Güvenliği: İmza Doğrulama

Bir webhook alıcısı (sizin sunucunuz), gelen POST isteğinin GERÇEKTEN beklenen kaynaktan geldiğini nasıl doğrular? Herhangi biri, sizin webhook URL'nizi bilse, sahte bir "ödeme tamamlandı" isteği gönderebilir! Çözüm: gönderen taraf (örn. Stripe), her istekle birlikte, gövdenin paylaşılan gizli bir anahtarla (shared secret) imzalanmış halini de bir başlıkta gönderir (Stripe-Signature gibi) — alıcı, gövdeyi kendi tarafında AYNI anahtarla yeniden imzalayıp, gelen imzayla EŞLEŞİP eşleşmediğini kontrol eder (bkz. 22.4.1, Bölüm 2.3'teki JWT imza doğrulamasıyla kavramsal olarak aynı prensip).

4.2. Webhook'ların Güvenilirliği: Yeniden Deneme ve İdempotentlik

Webhook alıcınızın sunucusu geçici olarak kapalıysa ya da zaman aşımına uğrarsa ne olur? İyi webhook sağlayıcıları (Stripe, GitHub), başarısız teslimatları OTOMATİK olarak, üstel geri çekilmeyle (bkz. 22.5.1, Bölüm 3.2) tekrar dener. Bu da alıcı tarafında ÖNEMLİ bir sonuç doğurur: AYNI webhook olayı birden fazla kez gelebilir — bu yüzden webhook işleyicileri İDEMPOTENT (bkz. 22.2, Bölüm 3.1) yazılmalıdır: her olayın benzersiz bir ID'si kontrol edilip, daha önce işlenmiş bir olay TEKRAR işlenmemelidir (örn. aynı ödeme iki kez "tamamlandı" olarak kaydedilmemelidir).

4.3. WebSocket'ler: Gerçek Zamanlı Çift Yönlü İletişim

Webhook'lar tek seferlik, sunucu-başlatan bildirimlerdir. Sürekli, ÇİFT YÖNLÜ gerçek zamanlı iletişim gerektiğinde (örn. bir canlı sohbet uygulaması, bir borsa fiyat akışı, çok oyunculu bir oyun) WebSocket protokolü kullanılır — HTTP ile başlayan ama sonra SÜREKLİ AÇIK kalan, hem istemcinin hem sunucunun istediği an mesaj gönderebildiği bir bağlantı kurar. FastAPI, `@app.websocket()` dekoratörüyle native

WebSocket desteęi sunar.

SONUÇ

REST'in over/under-fetching sorununu GraphQL'in nasıl çözdüğünü, yüksek performanslı mikroservis-içi iletişim için gRPC'nin neden tercih edildiğini ve API'lerin "sor-cevapla" modelinin ötesinde webhook'lar ve WebSocket'lerle nasıl OLAY TABANLI/gerçek zamanlı hale gelebileceğini öğrendik. Modülün son teorik dosyasında (22.9.1), tek bir API'nin ötesine geçip, GERÇEK bir şirketin onlarca API'sini nasıl YÖNETTİĞİNİ ve ÖLÇEKLENDİRDİĞİNİ inceleyeceğiz.